



School of Computer Engineering
Computer Engineering
Operations Research

Project 00

Knapsack Problem Analysis-Execution times and
success rates

Dayron Josué Porras-Pérez
2023124693

Javier Lee-Liang
2020184824

José Julián Araya-Castillo
2020098909

San José, Costa Rica
September 2026

Contents

1	Introduction	2
2	Execution Time Growth with Respect to the Number of Items and Knapsack Capacity	3
2.1	Dynamic Programming	3
2.2	Greedy Algorithms	3
3	Comparison Between Basic and Proportional Greedy Algorithms	5
4	Greedy Accuracy with Respect to the Optimal Solution	7
4.1	Basic Greedy Accuracy	7
4.2	Proportional Greedy Accuracy	8
5	Conclusions	9

Chapter 1

Introduction

This report analyzes the behavior of different algorithms used to solve the Knapsack Problem. Specifically, it evaluates the growth of execution times as the number of items and the knapsack capacity increase. Additionally, it compares the accuracy of greedy approaches with respect to the optimal solution.

The study focuses on two main aspects:

- Execution time growth for Dynamic Programming and Greedy algorithms.
- Accuracy percentage of Greedy algorithms compared to the optimal solution.

Chapter 2

Execution Time Growth with Respect to the Number of Items and Knapsack Capacity

2.1 Dynamic Programming

In the case of Dynamic Programming, the execution time increases as the number of items in the problem grows. Likewise, increasing the knapsack capacity also produces an increase in execution time.

However, the growth associated with the number of items is significantly larger than the growth associated with knapsack capacity. This behavior can be observed because the number of items increases in increments of ten, while the knapsack capacity increases in increments of one hundred. Even though execution times increase in both cases, the increase is proportionally more sensitive to the number of items.

The results are illustrated in the table 2.1.

2.2 Greedy Algorithms

For Greedy algorithms, the execution time growth between test cases is considerably smaller. This occurs because greedy approaches do not need to analyze and solve multiple subproblems. Instead, they simply traverse the list of available items and make local decisions.

Dynamic Programming, on the other hand, evaluates many subproblems in order to obtain the optimal solution, which results in higher computational costs.

Knapsack	Amount of items									
	10	20	30	40	50	60	70	80	90	100
100	0.0123	0.0267	0.0368	0.0542	0.0660	0.0802	0.0917	0.1069	0.1122	0.1273
200	0.0240	0.0478	0.0740	0.1031	0.1267	0.1540	0.1820	0.2108	0.2288	0.2568
300	0.0350	0.0737	0.1096	0.1456	0.1892	0.2298	0.2740	0.3129	0.3411	0.3886
400	0.0491	0.1001	0.1474	0.1980	0.2507	0.3069	0.3559	0.4146	0.4591	0.5042
500	0.0593	0.1224	0.1860	0.2485	0.3221	0.3808	0.4720	0.5178	0.6006	0.6393
600	0.0761	0.1465	0.2189	0.3015	0.3815	0.4590	0.5379	0.6198	0.6823	0.7746
700	0.0829	0.1732	0.2652	0.3477	0.4395	0.5319	0.6240	0.7305	0.7907	0.9062
800	0.0947	0.1985	0.2973	0.4004	0.5079	0.6077	0.7498	0.8285	0.9127	1.0371
900	0.1091	0.2229	0.3315	0.4540	0.6248	0.7106	0.8142	0.9342	1.0228	1.1438
1000	0.1198	0.2454	0.3728	0.5038	0.6398	0.7647	0.9199	1.0425	1.1858	1.2750

Figure 2.1: Dynamic Programming

Despite this difference, both approaches exhibit the same general trend: execution times grow more rapidly with respect to the number of items than with respect to knapsack capacity.

Chapter 3

Comparison Between Basic and Proportional Greedy Algorithms

The Basic Greedy algorithm exhibits even lower execution-time growth than the Proportional Greedy algorithm.

Knapsack	Amount of items									
	10	20	30	40	50	60	70	80	90	100
100	0.0011	0.0020	0.0032	0.0046	0.0063	0.0072	0.0122	0.0114	0.0120	0.0132
200	0.0010	0.0020	0.0031	0.0052	0.0066	0.0093	0.0091	0.0116	0.0141	0.0139
300	0.0011	0.0021	0.0035	0.0048	0.0065	0.0076	0.0104	0.0174	0.0137	0.0140
400	0.0009	0.0020	0.0033	0.0047	0.0071	0.0082	0.0104	0.0117	0.0130	0.0138
500	0.0010	0.0021	0.0031	0.0056	0.0064	0.0087	0.0091	0.0120	0.0131	0.0148
600	0.0010	0.0021	0.0032	0.0049	0.0070	0.0092	0.0094	0.0117	0.0128	0.0152
700	0.0010	0.0023	0.0035	0.0047	0.0068	0.0081	0.0102	0.0115	0.0139	0.0157
800	0.0009	0.0022	0.0036	0.0047	0.0064	0.0075	0.0101	0.0121	0.0131	0.0197
900	0.0010	0.0024	0.0033	0.0051	0.0095	0.0086	0.0104	0.0121	0.0129	0.0148
1000	0.0009	0.0022	0.0036	0.0054	0.0075	0.0082	0.0104	0.0131	0.0130	0.0157

Figure 3.1: Basic Greedy

Knapsack	Amount of items									
	10	20	30	40	50	60	70	80	90	100
100	0.0011	0.0027	0.0045	0.0077	0.0081	0.0110	0.0123	0.0141	0.0159	0.0192
200	0.0011	0.0023	0.0039	0.0054	0.0085	0.0092	0.0105	0.0134	0.0149	0.0165
300	0.0016	0.0031	0.0040	0.0052	0.0072	0.0088	0.0107	0.0114	0.0162	0.0166
400	0.0010	0.0022	0.0041	0.0049	0.0071	0.0083	0.0097	0.0121	0.0130	0.0142
500	0.0008	0.0020	0.0034	0.0044	0.0063	0.0075	0.0091	0.0113	0.0120	0.0140
600	0.0008	0.0017	0.0037	0.0043	0.0066	0.0081	0.0086	0.0126	0.0114	0.0135
700	0.0006	0.0016	0.0030	0.0041	0.0062	0.0071	0.0088	0.0096	0.0110	0.0129
800	0.0008	0.0020	0.0030	0.0042	0.0057	0.0069	0.0092	0.0098	0.0106	0.0160
900	0.0008	0.0017	0.0027	0.0040	0.0067	0.0065	0.0089	0.0099	0.0098	0.0116
1000	0.0008	0.0016	0.0028	0.0038	0.0051	0.0066	0.0080	0.0095	0.0096	0.0125

Figure 3.2: Proportional Greedy

Chapter 4

Greedy Accuracy with Respect to the Optimal Solution

4.1 Basic Greedy Accuracy

As the number of items increases, the Basic Greedy algorithm shows a considerable decrease in its success rate. It can be observed that in most cases, once the number of items exceeds forty, the algorithm frequently fails to find the optimal solution.

The following table illustrates the percentage of successful matches with the optimal solution.

Knapsack	Amount of items									
	10	20	30	40	50	60	70	80	90	100
100	74.00%	13.00%	0.00%	2.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
200	54.00%	11.00%	2.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
300	69.00%	16.00%	2.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
400	65.00%	13.00%	2.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
500	62.00%	10.00%	3.00%	2.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
600	64.00%	10.00%	2.00%	2.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
700	63.00%	16.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
800	53.00%	13.00%	4.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
900	71.00%	9.00%	2.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%
1000	63.00%	14.00%	3.00%	1.00%	0.00%	0.00%	0.00%	0.00%	0.00%	0.00%

Figure 4.1: Basic Greedy Accuracy Table

4.2 Proportional Greedy Accuracy

For the Proportional Greedy algorithm, the decrease in accuracy occurs more gradually. Nevertheless, the percentage of correct solutions still declines as the number of items increases.

Therefore, larger problem instances tend to reduce the probability that the proportional greedy strategy will produce the optimal solution.

The following table presents the corresponding accuracy percentages.

Knapsack	Amount of items									
	10	20	30	40	50	60	70	80	90	100
100	74.00%	64.00%	43.00%	51.00%	47.00%	38.00%	38.00%	42.00%	38.00%	41.00%
200	76.00%	48.00%	46.00%	35.00%	36.00%	37.00%	40.00%	32.00%	30.00%	30.00%
300	77.00%	61.00%	27.00%	17.00%	22.00%	26.00%	28.00%	21.00%	25.00%	24.00%
400	79.00%	49.00%	43.00%	30.00%	19.00%	14.00%	10.00%	8.00%	10.00%	10.00%
500	68.00%	24.00%	24.00%	39.00%	33.00%	15.00%	27.00%	14.00%	6.00%	4.00%
600	70.00%	18.00%	11.00%	17.00%	22.00%	19.00%	35.00%	31.00%	23.00%	15.00%
700	69.00%	24.00%	8.00%	4.00%	5.00%	7.00%	13.00%	21.00%	13.00%	20.00%
800	57.00%	18.00%	7.00%	1.00%	3.00%	4.00%	2.00%	3.00%	5.00%	9.00%
900	71.00%	13.00%	8.00%	3.00%	0.00%	0.00%	1.00%	2.00%	2.00%	3.00%
1000	65.00%	17.00%	4.00%	2.00%	0.00%	0.00%	0.00%	0.00%	2.00%	0.00%

Figure 4.2: Proportional Greedy Accuracy Table

Chapter 5

Conclusions

1. Dynamic Programming experiences a significant increase in execution time as the number of items grows.
2. Knapsack capacity also affects execution time, although its impact is smaller than the impact of increasing the number of items.
3. Greedy algorithms are computationally more efficient because they do not solve subproblems.
4. The Basic Greedy algorithm experiences a rapid decrease in accuracy as problem size increases.
5. The Proportional Greedy algorithm maintains higher accuracy than the Basic Greedy algorithm, but its success rate also decreases as the number of items grows.
6. Dynamic Programming provides the optimal solution at the cost of higher execution times, whereas Greedy algorithms offer faster execution with reduced optimality guarantees.